

Лабораторна робота №2

Багатопоковість. Асинхронність. IEnumerable. LINQ.

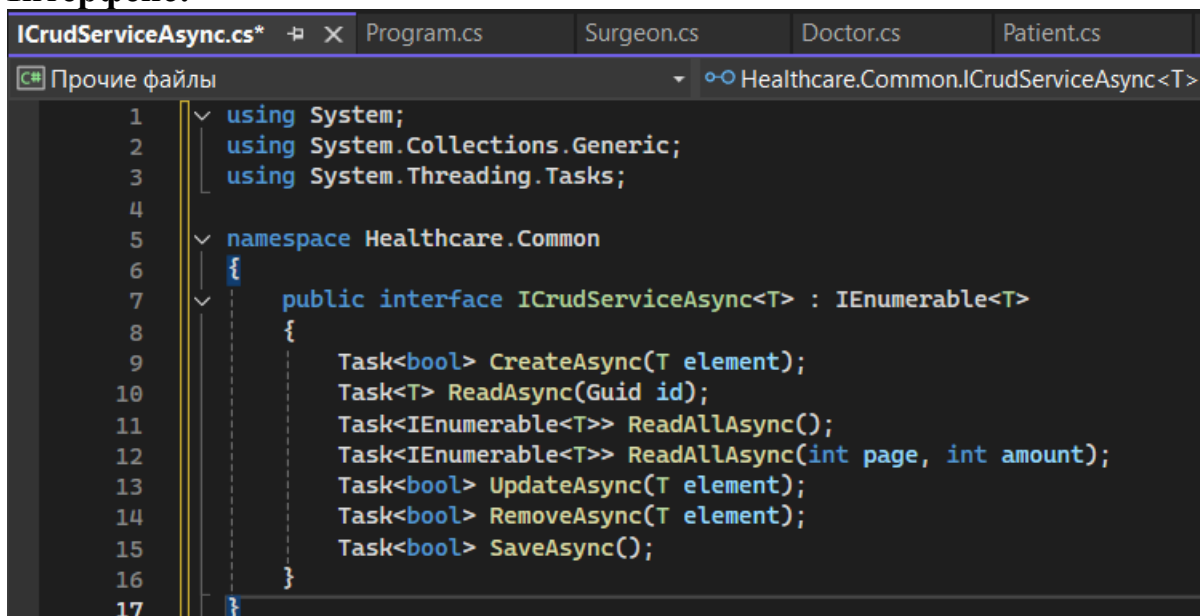
Виконав студент групи 302-ТК
Зройчиков Вячеслав

Завдання

1. Реалізувати асинхронну версію дженерік CRUD сервісу, який буде зберігати дані у одній із вбудованих колекцій .NET, буде багатопотоково-безпечною (thread safe), зберігатиме асинхронно колекцію у серіалізованому вигляді у файлі за шляхом FilePath, матиме вбудовану підтримку пагінації та реалізовуватиме наступний інтерфейс та інтерфейс IEnumerable:

```
public interface ICrudServiceAsync<T> : IEnumerable<T>
{
    public Task<bool> CreateAsync(T element);
    public Task<T> ReadAsync(Guid id);
    public Task<IEnumerable<T>> ReadAllAsync();
    public Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
    public Task<bool> UpdateAsync(T element);
    public Task<bool> RemoveAsync(T element);
    public Task<bool> SaveAsync();
}
```

Інтерфейс:



```
ICrudServiceAsync.cs*  Program.cs  Surgeon.cs  Doctor.cs  Patient.cs
C# Прочие файлы  Healthcare.Common.ICrudServiceAsync<T>
1  using System;
2  using System.Collections.Generic;
3  using System.Threading.Tasks;
4
5  namespace Healthcare.Common
6  {
7      public interface ICrudServiceAsync<T> : IEnumerable<T>
8      {
9          Task<bool> CreateAsync(T element);
10         Task<T> ReadAsync(Guid id);
11         Task<IEnumerable<T>> ReadAllAsync();
12         Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
13         Task<bool> UpdateAsync(T element);
14         Task<bool> RemoveAsync(T element);
15         Task<bool> SaveAsync();
16     }
17 }
```

Реалізація:

```
C# Прочие файлы Healthc
1 using System;
2 using System.Collections;
3 using System.Collections.Concurrent;
4 using System.Collections.Generic;
5 using System.IO;
6 using System.Linq;
7 using System.Text.Json;
8 using System.Threading.Tasks;
9
10 namespace Healthcare.Common
11 {
12     public class CrudServiceAsync<T> : ICrudServiceAsync<T> where T : class
13     {
14         private readonly ConcurrentDictionary<Guid, T> _storage = new();
15         private readonly Func<T, Guid> _keySelector;
16         public string FilePath { get; }
17
18         public CrudServiceAsync(string filePath, Func<T, Guid> keySelector)
19         {
20             FilePath = filePath;
21             _keySelector = keySelector ?? throw new ArgumentNullException(nameof(keySelector));
22         }
23
24         public Task<bool> CreateAsync(T element)
25         {
26             var id = _keySelector(element);
27             if (_storage.ContainsKey(id)) return Task.FromResult(false);
28             return Task.FromResult(_storage.TryAdd(id, element));
29         }
30
31         public Task<T> ReadAsync(Guid id)
32         {
33             _storage.TryGetValue(id, out var item);
34             return Task.FromResult(item);
35         }
36
37         public Task<IEnumerable<T>> ReadAllAsync()
38         {
39             return Task.FromResult(_storage.Values.AsEnumerable());
40         }
41
42         public Task<IEnumerable<T>> ReadAllAsync(int page, int amount)
43         {
44             return Task.FromResult(_storage.Values.Skip((page - 1) * amount).Take(amount));
45         }
46
47         public Task<bool> UpdateAsync(T element)
48         {
49             var id = _keySelector(element);
50             if (!_storage.ContainsKey(id)) return Task.FromResult(false);
51             _storage[id] = element;
52             return Task.FromResult(true);
53         }
54
55         public Task<bool> RemoveAsync(T element)
56         {
57             var id = _keySelector(element);
58             return Task.FromResult(_storage.TryRemove(id, out _));
59         }
60
61         public async Task<bool> SaveAsync()
62         {
63             try
64             {
65                 var json = JsonSerializer.Serialize(_storage.Values);
66                 await File.WriteAllTextAsync(FilePath, json);
67                 return true;
68             }
69             catch
70             {
71                 return false;
72             }
73         }
74
75         public IEnumerator<T> GetEnumerator() => _storage.Values.GetEnumerator();
76         IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
77     }
78 }
```

Консольна програма:

```
CrudServiceAsync.cs* Program.cs Surgeon.cs Doctor.cs Patient.cs
Healthcare Healthcare.Program
1 using System;
2 using System.Threading.Tasks;
3 using Healthcare.Common;
4
5 namespace Healthcare
6 {
7     class Program
8     {
9         static async Task Main(string[] args)
10         {
11             // Створюємо сервіс CRUD з keySelector (як отримати ключ з Patient)
12             var patientService = new CrudServiceAsync<Patient>(
13                 "patients.json",
14                 patient => Guid.TryParse(patient.PatientId, out var id) ? id : Guid.Empty
15             );
16
17             // Створюємо нового пацієнта з унікальним PatientId (він має бути Guid!)
18             var newPatient = new Patient
19             {
20                 Name = "Anna Smith",
21                 Gender = "Female",
22                 Age = 30,
23                 PatientId = Guid.NewGuid().ToString(),
24                 Diagnosis = "Flu",
25                 InsuranceId = "INS-12345"
26             };
27
28             // Створення
29             var created = await patientService.CreateAsync(newPatient);
30             Console.WriteLine(created ? "Patient created!" : "Patient already exists!");
31
32             // Читання
33             var patientFromDb = await patientService.ReadAsync(Guid.Parse(newPatient.PatientId));
34             Console.WriteLine($"Found: {patientFromDb?.GetInfo()}");
35
36             // Оновлення
37             newPatient.Diagnosis = "Recovered";
38             await patientService.UpdateAsync(newPatient);
39
40             // Парінація
41             var page1 = await patientService.ReadAllAsync(1, 10);
42             Console.WriteLine($"Patients on page 1:");
43             foreach (var p in page1)
44             {
45                 Console.WriteLine(p.GetInfo());
46             }
47
48             // Збереження до файлу
49             var saved = await patientService.SaveAsync();
50             Console.WriteLine(saved ? "Saved to file." : "Failed to save.");
51         }
52     }
53 }
```

Результат:

```
Консоль отладки Microsoft Visual Studio

? Дан? завантажено з файлу: patients.json
?? Ус? пац?єнти:
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Pneumonia, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Cold, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003
Ivan Ivanov, Male, Age: 35, ID: P001, Diagnosis: Flu, Insurance: INS001
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Cold, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003

?? П?сля оновлення:
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Pneumonia, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Cold, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003
Ivan Ivanov, Male, Age: 35, ID: P001, Diagnosis: Flu, Insurance: INS001
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Cold, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003

?? П?сля видалення:
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Pneumonia, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Cold, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Cold, Insurance: INS002
Sergey Sidorov, Male, Age: 42, ID: P003, Diagnosis: Diabetes, Insurance: INS003

?? Знайдено пац?єнта:
Anna Petrovna, Female, Age: 28, ID: P002, Diagnosis: Pneumonia, Insurance: INS002
? Дан? збережено у файл: patients.json

Натисн?ть будь-яку клаву для завершення...
```

Інтерфейс базової сутності:

```
Прочие файлы
1 using System;
2
3 namespace Healthcare.Common
4 {
5     public interface IEntity
6     {
7         Guid Id { get; }
8     }
9 }
```

- Створіть статичні методи у класах, які створюватимуть нові об'єкти цього класу із згенерованими даними:

```
public class Bus
{
    public static Bus CreateNew()
    {
        throw new NotImplementedException();
    }
}
```

```
Healthcare.Common Healthcare.Common.Person
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace Healthcare.Common
8 {
9     public class Person
10     {
11         public string FullName { get; set; }
12         public string Gender { get; set; }
13         public int Age { get; set; }
14
15         // Статичне поле
16         public static int PersonCount;
17
18         // Статичний конструктор
19         static Person()
20         {
21             PersonCount = 0;
22         }
23
24         // Конструктор
25         public Person(string fullName, string gender, int age)
26         {
27             FullName = fullName;
28             Gender = gender;
29             Age = age;
30             PersonCount++;
31         }
32
33         public static Person CreateNew()
34         {
35             var random = new Random();
36             string[] names = { "Ivan Petrenko", "Olga Tkachenko", "Nazar Bondar" };
37             string[] genders = { "Male", "Female" };
38
39             return new Person(
40                 names[random.Next(names.Length)],
41                 genders[random.Next(genders.Length)],
42                 random.Next(18, 80)
43             );
44         }
45
46         // Метод
47         public string GetInfo()
48         {
49             return $"{FullName}, {Gender}, {Age} years old";
50         }
51
52         // Статичний метод
53         public static int GetTotalPeople()
54         {
55             return PersonCount;
56         }
57     }
58 }
```

Використання:

```
var newPatient = Patient.CreateNew();
```

```
Console.WriteLine(newPatient.GetInfo());
```

```
var newSurgeon = Surgeon.CreateNew();
```

```
Console.WriteLine(newSurgeon.GetStats());
```

3. Модифікуйте консольний застосунок, щоб він паралельно створив від тисячі об'єктів використовуючи CRUD service, створені у об'єктах метод CreateNew та клас Parallell. Для створених об'єктів знайдіть мінімальні, максимальні та середні значення для цифрових значень, отримані результати виведіть у консоль. Згенеровану колекцію збережіть у файл.

Консольна програма:

```
Healthcare Program.cs
Healthcare Healthcare.Program
7 using System.IO;
8 using Healthcare.Common;
9
10 namespace Healthcare
11 {
12     class Program
13     {
14         static async Task Main(string[] args)
15         {
16             Console.WriteLine("Генерація 1000 пацієнтів у паралельному режимі...");
17
18             var service = new CrudServiceAsync<Patient>(
19                 "patients_generated.json",
20                 patient => Guid.TryParse(patient.PatientId, out var id) ? id : Guid.NewGuid()
21             );
22
23             int count = 1000;
24             var options = new ParallelOptions { MaxDegreeOfParallelism = Environment.ProcessorCount };
25             var bag = new ConcurrentBag<Patient>();
26
27             Parallel.For(0, count, options, i =>
28             {
29                 var patient = Patient.CreateNew();
30                 bag.Add(patient);
31             });
32
33             Console.WriteLine("Додавання пацієнтів до сервісу...");
34
35             var tasks = bag.Select(p => service.CreateAsync(p));
36             await Task.WhenAll(tasks);
37
38             var allPatients = await service.ReadAllAsync();
39
40             if (allPatients == null || !allPatients.Any())
41             {
42                 Console.WriteLine("Жодного пацієнта не знайдено.");
43                 return;
44             }
45
46             var ages = allPatients.Select(p => p.Age).ToList();
47             int minAge = ages.Min();
48             int maxAge = ages.Max();
49             double avgAge = ages.Average();
50
51             Console.WriteLine("\nСтатистика за віком:");
52             Console.WriteLine($"Мінімальний вік: {minAge}");
53             Console.WriteLine($"Максимальний вік: {maxAge}");
54             Console.WriteLine($"Середній вік: {avgAge:F2}");
55
56             Console.WriteLine("\nЗбереження даних у файл...");
57
58             if (await service.SaveAsync())
59                 Console.WriteLine("Колекцію успішно збережено у файл.");
60             else
61                 Console.WriteLine("Помилка при збереженні колекції.");
62         }
63     }
64 }
65
```

Результат:

```
Консоль отладки Microsoft V
?? Генерація 1000 пацієнтів у паралельному режимі...
?? Додавання пацієнтів до сервісу...

?? Статистика за віком:
?? Мінімальний вік: 18
?? Максимальний вік: 89
?? Середній вік: 54,69

?? Збереження даних у файл...
? Колекцію успішно збережено у файл.

E:\NUPP_NET_2025_302_tk_Zroychikov_Lab\Healthcare\bin\Debug\net9.0\Healthcare.exe (процесс 18688) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

4. Додайте приклади використання примітивів синхронізації як Lock, Semaphore, AutoResetEvent та інші.

Додаємо до класу Patient; Lock, Semaphore, AutoResetEvent:

```
57 // === Прімітивні синхронізації ===
58
59 private static object lockObj = new object();
60 private static int sharedCounter = 0;
61
62 Ссылка 1
63 public static void LockExample()
64 {
65     void Increment()
66     {
67         for (int i = 0; i < 1000; i++)
68         {
69             lock (lockObj)
70             {
71                 sharedCounter++;
72             }
73         }
74
75         Thread t1 = new Thread(Increment);
76         Thread t2 = new Thread(Increment);
77         t1.Start(); t2.Start();
78         t1.Join(); t2.Join();
79
80         Console.WriteLine($"LockExample: sharedCounter = {sharedCounter}");
81     }
82
83     private static Semaphore semaphore = new Semaphore(2, 2);
84
85     Ссылка 1
86     public static void SemaphoreExample()
87     {
88         void Access(int id)
89         {
90             Console.WriteLine($"Потік {id} чекає доступ...");
91             semaphore.WaitOne();
92             Console.WriteLine($"Потік {id} працює...");
93             Thread.Sleep(1000);
94             Console.WriteLine($"Потік {id} завершує.");
95             semaphore.Release();
96         }
97
98         for (int i = 0; i < 5; i++)
99         {
100             int localId = i;
101             new Thread(() => Access(localId)).Start();
102         }
103
104         private static AutoResetEvent autoEvent = new AutoResetEvent(false);
105
106         Ссылка 1
107         public static void AutoResetEventExample()
108         {
109             new Thread(() =>
110             {
111                 Console.WriteLine("AutoResetEvent: Потік чекає...");
112                 autoEvent.WaitOne();
113                 Console.WriteLine("AutoResetEvent: Отримано сигнал!");
114             }).Start();
115             Thread.Sleep(1000);
116         }
117     }
```

```

        Console.WriteLine("AutoResetEvent: Відправка сигналу...");
        autoEvent.Set();
    }

    private static ManualResetEvent manualEvent = new ManualResetEvent(false);

    Ссылка 1
    public static void ManualResetEventExample()
    {
        for (int i = 0; i < 3; i++)
        {
            new Thread(() =>
            {
                Console.WriteLine("ManualResetEvent: Потік чекає...");
                manualEvent.WaitOne();
                Console.WriteLine("ManualResetEvent: Потік продовжив роботу.");
            }).Start();

            Thread.Sleep(1000);
            Console.WriteLine("ManualResetEvent: Надсилається сигнал...");
            manualEvent.Set(); // Всі потоки продовжують
        }
    }

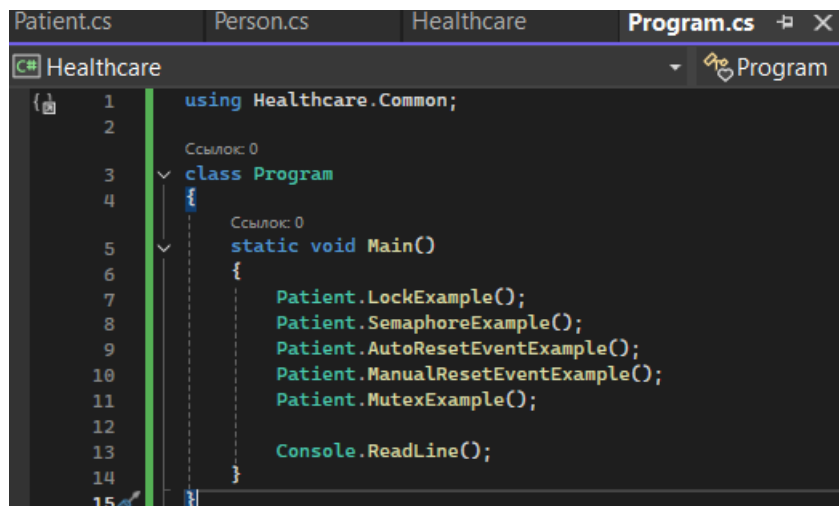
    private static Mutex mutex = new Mutex();

    Ссылка 1
    public static void MutexExample()
    {
        void Work(int id)
        {
            Console.WriteLine($"Mutex: Потік {id} чекає...");
            mutex.WaitOne();
            Console.WriteLine($"Mutex: Потік {id} працює...");
            Thread.Sleep(500);
            Console.WriteLine($"Mutex: Потік {id} завершує.");
            mutex.ReleaseMutex();
        }

        for (int i = 0; i < 3; i++)
        {
            int localId = i;
            new Thread(() => Work(localId)).Start();
        }
    }
}

```

Використання:



```

Patient.cs  Person.cs  Healthcare  Program.cs
C# Healthcare
1  using Healthcare.Common;
2
3  Ссылка 0
4  class Program
5  {
6      Ссылка 0
7      static void Main()
8      {
9          Patient.LockExample();
10         Patient.SemaphoreExample();
11         Patient.AutoResetEventExample();
12         Patient.ManualResetEventExample();
13         Patient.MutexExample();
14
15         Console.ReadLine();
16     }
17 }

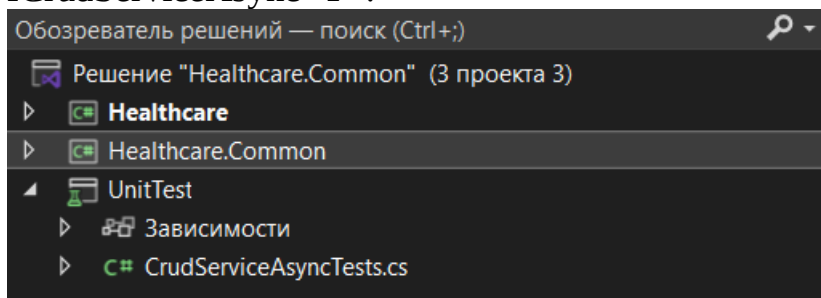
```

Результат:

```
E:\NUPP_NET_2025_302_tk_Zr  ×  +  v
LockExample: sharedCounter = 2000
Пот?к 1 чекає доступ...
Пот?к 2 чекає доступ...
Пот?к 0 чекає доступ...
Пот?к 1 працює...
Пот?к 2 працює...
Пот?к 3 чекає доступ...
Пот?к 4 чекає доступ...
AutoResetEvent: Пот?к чекає...
AutoResetEvent: В?дправка сигналу...
Пот?к 2 завершує.
Пот?к 1 завершує.
AutoResetEvent: Отримано сигнал!
Пот?к 0 працює...
Пот?к 3 працює...
ManualResetEvent: Пот?к чекає...
ManualResetEvent: Пот?к чекає...
ManualResetEvent: Пот?к чекає...
ManualResetEvent: Надсилається сигнал...
Пот?к 3 завершує.
Пот?к 0 завершує.
ManualResetEvent: Пот?к продовжив роботу.
ManualResetEvent: Пот?к продовжив роботу.
ManualResetEvent: Пот?к продовжив роботу.
Пот?к 4 працює...
Mutex: Пот?к 1 чекає...
Mutex: Пот?к 1 працює...
Mutex: Пот?к 0 чекає...
Mutex: Пот?к 2 чекає...
Mutex: Пот?к 1 завершує.
Mutex: Пот?к 0 працює...
Пот?к 4 завершує.
Mutex: Пот?к 0 завершує.
Mutex: Пот?к 2 працює...
Mutex: Пот?к 2 завершує.
```

Додаткове Завдання

Створіть модульні (Unit) тести для створеною вами реалізації `ICrudServiceAsync<T>`.



Клас:

```
1  using System;
2  using System.Collections.Generic;
3  using System.Threading.Tasks;
4  using Healthcare.Common;
5  using Moq;
6  using Xunit;
7
8  namespace Healthcare.Tests
9  {
10     Ссылка: 1
11     public class CrudServiceAsyncTests
12     {
13         private readonly Mock<ICrudServiceAsync<Patient>> _mockService;
14         private readonly Patient _testPatient;
15
16         Ссылка: 0
17         public CrudServiceAsyncTests()
18         {
19             _mockService = new Mock<ICrudServiceAsync<Patient>>();
20
21             _testPatient = new Patient
22             {
23                 Name = "Test Patient",
24                 Age = 45,
25                 Gender = "Female",
26                 PatientId = Guid.NewGuid().ToString(),
27                 Diagnosis = "Cold",
28                 InsuranceId = "INS001"
29             };
30
31         }
32
33         Ссылка: 0
34         [Fact]
35         public async Task CreateAsync_ReturnsTrue()
36         {
37             _mockService.Setup(s => s.CreateAsync(_testPatient)).ReturnsAsync(true);
38
39             var result = await _mockService.Object.CreateAsync(_testPatient);
40
41             Assert.True(result);
42         }
43
44         Ссылка: 0
45         [Fact]
46         public async Task ReadAsync_ReturnsPatient()
47         {
48             var id = Guid.NewGuid();
49             _mockService.Setup(s => s.ReadAsync(id)).ReturnsAsync(_testPatient);
50
51             var result = await _mockService.Object.ReadAsync(id);
52
53             Assert.NotNull(result);
54         }
55     }
56 }
```

```

51
52 [Fact]
   Ссылка: 0
53 public async Task ReadAllAsync_ReturnsList()
54 {
55     var list = new List<Patient> { _testPatient };
56     _mockService.Setup(s => s.ReadAllAsync()).ReturnsAsync(list);
57
58     var result = await _mockService.Object.ReadAllAsync();
59
60     Assert.Single(result);
61 }
62
63 [Fact]
   Ссылка: 0
64 public async Task ReadAllPaged_ReturnsPagedList()
65 {
66     var paged = new List<Patient> { _testPatient };
67     _mockService.Setup(s => s.ReadAllAsync(1, 1)).ReturnsAsync(paged);
68
69     var result = await _mockService.Object.ReadAllAsync(1, 1);
70
71     Assert.Single(result);
72 }
73
74 [Fact]
   Ссылка: 0
75 public async Task UpdateAsync_ReturnsTrue()
76 {
77     _mockService.Setup(s => s.UpdateAsync(_testPatient)).ReturnsAsync(true);
78
79     var result = await _mockService.Object.UpdateAsync(_testPatient);
80
81     Assert.True(result);
82 }
83
84 [Fact]
   Ссылка: 0
85 public async Task RemoveAsync_ReturnsTrue()
86 {
87     _mockService.Setup(s => s.RemoveAsync(_testPatient)).ReturnsAsync(true);
88
89     var result = await _mockService.Object.RemoveAsync(_testPatient);
90
91     Assert.True(result);
92 }
93
94 [Fact]
   Ссылка: 0
95 public async Task SaveAsync_ReturnsTrue()

```

Результат:

Консоль отладки Microsoft Visual Studio

```

.\Program Files\dotnet\dotnet.exe (процесс 19688) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:

```